

KittyPatcher

v0.7.6 Modder Guide

Course of Temptation | Writing Mods, Porting Mods, Debugging Failures

Introduction

This guide is for mod authors writing or maintaining .mod files for Course of Temptation. It covers every directive in full depth, how resilient each one is to game updates, the modding strategy that minimizes porting work, built-in tools that diagnose and fix failures after a game update, and all v0.7.4 changes relevant to mod authors.

The Modder Edition ships with KittyDiffAssist.py (generates detailed failure reports and auto-ported draft mod files when any Replace: block fails) and KittyModConverter.py (scans existing mods and suggests which Replace: blocks could be replaced with more robust directives).

v0.7.0 modder changes summary: dependency graph load ordering via cm_dependencies.json; structured mod header fields (Mod Name:, Version:, Author:, Requires:, Priority:); Hook around: timing; two new guards (IfPassageContains, IfPassageHasTag); With: auto-escape; patch result cache; BeautifulSoup 4 HTML layer; and ST_UNSCOPED advisory warning in KittyValidate.

v0.7.4 modder changes summary: content-addressed SHA-256 patch cache (startup ~4 min to ~0.2 s on hit); batch pre-pass for Append To Passage and Insert After/Before; two-pass Replace: anchor search fixing silent failure on JS script-block targets; auto-close after browser launch; Hook conflict fix (second+ mod hooks now fire); three bug fixes (cache on failure, cache key, duplicate Hook handling).

BEST PRACTICE: Put your logic in Tier S directives (Add Function, Add Passage). Call it from Tier 2 (Hook, Insert After). Use Replace: only when you genuinely need to modify vanilla behavior inline.

1. Directive Resilience Hierarchy

Every directive is assigned a porting cost -- how much work is needed to keep it working after a game update. Always choose the most resilient directive that can do the job.

Tier	Directives	Porting cost / anchor
S	Add Passage:, Add Javascript:, Add CSS:, Add Events:, Add Function, Add Variable, Rename Passage, Move Passage, Add/Remove Tag To Passage	Zero -- patcher never touches existing code
1	Prepend To Passage, Append To Passage	Zero -- anchors on passage name only, injects at body edges
2	Hook [name]:, Insert Before [target]:, Insert After [target]:	Near-zero -- anchors on function name or short stable string
2.5	Insert Into Function, Replace In Function, Replace Function Signature, Delete In Passage	Low-moderate -- anchors on name + anchor within body
3	Replace Function [name]:	Moderate -- name resilient, but you own full body silently
4	Replace In Passage, Delete Span	Same as Replace: cost but scope-isolated to one passage
5	ReplaceReg [pattern]:, Replace: / With:	Highest -- every line of target is a break point
S	Append Array, Append Var/Func to Class, Insert Into Array/Object, Delete Block	Low -- anchors on name

Tier S: Version-proof permanently. Tier 1: Anchors on passage name only. Tier 2: Fails only if the game renames a function or removes an anchor string entirely. Tier 2.5: Resilient to changes outside the target scope; breaks if anchor within scoped body changes. Tier 3: Function name is resilient but you silently own the full function body -- dev changes discarded. Tier 4-5: Every targeted line is a potential break point. Minimize target length.

2. Additive Directives (Tier S)

These directives never modify existing game code and never need porting.

Add Passage:

Injects a new `<tw-passagedata>` block into the story. Skipped automatically if a passage with the same name already exists, making it safe to run multiple times.

Full format

```
Add Passage:
<tw-passagedata pid="70001" name="MyModPassage" tags="noevents nobr">
<<set $myvar to true>>
```

```
Some content here.
</tw-passagedata>
```

Use pid="auto" for auto-assignment during PID renormalization. Raw Twine syntax (<<>> macros) is auto-escaped by the patcher. Duplicate passage names across mods are logged to FailsPatchLog.txt; the second copy is silently skipped.

WARNING: If two mods define a passage with the same name, the second is silently dropped. This is not a conflict warning -- it is a skip with a log entry. Check FailsPatchLog.txt after every run.

Shorthand format

```
Add Passage: MyModPassage
tags: noevents nobr
<<set $myvar to true>>
Some content here.
```

No <tw-passagedata> wrapper or closing tag needed. The patcher builds the full passage tag using pid="auto". The tags: line is optional. Body ends at the next directive or end of file.

Add Javascript:

Appends arbitrary JavaScript to the end of the game's script block. Use for config objects, event listeners, or any setup code that does not need a named function. For named functions, prefer Add Function.

```
Add Javascript:
setup.myConfig = {
  enabled: true,
  chance: 0.1,
};
$(document).on(":passageend", function() {
  setup.myConfig.enabled && setup.myCheck();
});
```

WARNING: Add Javascript: has no duplicate check. Wrap stateful setup code in a guard: if (!setup.myConfig) { ... }. Running the patcher twice without this guard will double-execute the setup code on page load, potentially corrupting game state.

Add CSS:

Appends CSS rules to the end of the game's style block. /* */ block comments are safe in CSS. DO NOT use // in CSS -- it is not valid CSS syntax and silently breaks all rules following it on that line.

```
Add CSS:
.cm-mymod-highlight {
  color: #ff6688;
  /* v1.0.0: initial style */
}
```

Add Events:

Appends entries to the game's event database array (setup.Events.db). The trailing comma on the last entry is required. Duplicate event passage registrations cause the event to fire ~Nx as often -- always audit mod files before release.

```
Add Events:
{
  passage: "MyEventPassage",
  tags: ["dorm", "gym"],
  chance: 0.05,
  conditions: [],
},
```

WARNING: Duplicate event entries (same passage name from two mods) cause the event to fire at approximately 2x intended frequency. Always check for duplicate event registrations when combining mods.

Add Function [name]:

Injects a new named JavaScript function into the script block. Skipped automatically if a function with that name already exists anywhere in the script block -- whether from the base game or a previous patch. Permanently version-proof for the function logic itself.

```
Add Function [setup.gangbang_check]:
setup.gangbang_check = function(config) {
  if (!config || !config.people) return false;
  return config.people.length >= 3 && State.variables.gangbangChance > 0;
};
```

BEST PRACTICE: Put all your JS logic in Add Function blocks, then call those functions from Hook or Insert After. The logic never needs porting. Only the call site anchor might shift.

Add Variable [name]: value

Injects <<set \$varname to value>>> into StoryInit. Skipped if the variable name already appears there. Saves a Prepend/Append To Passage for simple declarations.

```
Add Variable [myModEnabled]: true
Add Variable [myModScore]: 0
```

Rename Passage / Move Passage / Tag ops

Move Passage [OldName] To [NewName]: renames the passage AND updates all goto, link, display, include, and passage: references throughout the entire HTML. More thorough than Rename Passage.

Add Tag To Passage / Remove Tag From Passage modify the tags= attribute by passage name, eliminating fragile Replace: patterns targeting passage opening tags.

```
Move Passage [EventTownWalkOld] To [EventTownWalkNew]:
Add Tag To Passage [ShowerStall]: noevents
Remove Tag From Passage [MyPassage]: oldtag
```

Recommended pattern: Add Function + Hook

```
// Step 1: Define your logic (zero porting cost, permanent)
Add Function [setup.mymod_check]:
setup.mymod_check = function(config) {
  return config && config.people && config.people.length >= 3;
};

Add Function [setup.mymod_trigger]:
```

```

setup.mymod_trigger = function(config) {
  State.variables.myModActive = true;
};

// Step 2: Hook the call site (near-zero porting cost)
Hook [setup.build_encounter]:
after:
if (setup.mymod_check(config)) setup.mymod_trigger(config);

// Generated wrapper output:
setup.mymod_check = function(config) { ... }; // permanent
setup.mymod_trigger = function(config) { ... }; // permanent
var setup_build_encounter__orig = setup.build_encounter;
setup.build_encounter = function(config) {
  var _result = setup_build_encounter__orig.apply(this, arguments);
  if (setup.mymod_check(config)) setup.mymod_trigger(config);
  return _result;
};

```

The Hook approach has zero porting cost. If setup.build_encounter changes signature or gains new parameters, your Hook still works -- it just wraps whatever version exists.

3. Injection Directives (Tier 1-2)

Prepend To Passage [name]: and Append To Passage [name]:

Injects the With: body at the very start (Prepend) or very end (Append) of a named passage body. Anchors on the passage name only -- no anchor string within the passage is needed. Safer than Insert Before/After when the passage first or last line drifts across game versions.

```

Append To Passage [Cheats]:
With:
<<link "My Mod Debug">>
<<goto "MyModDebug">>
<</link>>

Prepend To Passage [StoryInit]:
With:
<<set $myModVersion to "1.0.0">>

```

Duplicate check: if the With: body is already present at the start/end, the block is skipped. When two Prepend directives target the same passage, both apply -- the second goes before the first.

Hook [name]: (v0.7.0 adds around: timing)

Wraps an existing named JavaScript function without replacing it. The original function is always called; your code runs before, after, or around (v0.7.0) it.

```

Hook [setup.people.generate]:
before:
console.log("About to generate an NPC");

Hook [setup.people.generate]:
after:

```

```

console.log("Generated:", _result);

Hook [setup.some_check]:
around:
var _result = setup_some_check__orig.apply(this,arguments);
if (_result && State.variables.myModActive) _result = _result * 1.5;
return _result;

```

WARNING: Only assignment-style functions (name = function(...) {}) are supported as hook targets. Hook is skipped if the wrapper variable (name __orig) already exists. _result is only available in after: and around: hooks -- do not reference it in before: hooks.

Insert Before [target]: and Insert After [target]:

Finds the first occurrence of the target string in the entire HTML and injects the With: body immediately before or after it. The target is a literal string, not regex. A duplicate check skips re-injection if the With: body is already present immediately adjacent to the target.

```

Insert After [Person.prototype.initStats = function() {}]:
With:
this.modStat = 0;

Insert Before [setup.Events.db =]:
With:
// Events registered by MyMod

```

WARNING: Insert Before and Insert After are global -- they match the FIRST occurrence of the target anywhere in the HTML. If the same anchor appears in multiple passages or functions, use Insert Into Function or Replace In Passage instead. KittyValidate v0.7.0 reports ST_UNSCOPED for these cases.

Insert Into Function [name]:

Scoped Insert After that only matches inside the named function body. Prevents false matches when the anchor string appears in multiple functions. The patcher locates the function by name using bracket-matching, finds the anchor within that body, and injects after it.

```

Insert Into Function [setup.build_encounter]:
var result = {};
With:
result.modFlag = true;

```

NOTE: Within Phase 4, Add Function is sorted before Insert Into Function. You can Insert Into Function a function that the same mod defines via Add Function.

4. Scoped Directives (Tier 2.5)

Replace In Function [name]:

Scoped replacement that replaces anchor text only within the named function body. The anchor only needs to be unique within that function, not the entire HTML. Prevents false matches when the same code snippet appears in multiple functions.

```

Replace In Function [setup.someFunc]:

```

```
return oldValue;
With:
return newValue + bonus;
```

Replace Function Signature [name]: newParams

Replaces only the parameter list of a named function, leaving the body completely intact. Useful when a mod needs to add new parameters without owning the full function body.

```
Replace Function Signature [setup.random_name]: name, ethnicity, traitassocid
```

Delete In Passage [name]:

Scoped deletion that removes anchor text from the named passage body only. No With: body needed. Write raw Twine in the anchor -- the patcher encodes it. If the passage is not found or the anchor is not found within it, the block fails and is logged.

```
Delete In Passage [ShowerStall]:
<<set _oldvar to true>>
```

Replace Function [name]: (Tier 3)

Replaces the entire body of an existing JavaScript function. The function signature is preserved; only the inner body is replaced. Write only the body contents -- not the surrounding braces.

```
Replace Function [setup.someExistingFunction]:
// New body
return doSomethingDifferent();
```

WARNING: You own the ENTIRE function body. Dev changes to that function in future updates are silently discarded. When two mods Replace Function the same target, a conflict is logged and the last applied wins. Prefer Hook + Add Function whenever possible.

5. Replace: / With: and ReplaceReg in Depth

The Replace: directive is the most powerful and the most fragile. Use it only when no Tier S, 1, 2, or 2.5 directive can accomplish the same result.

Writing a correct Replace: anchor

Open the game HTML in a text editor. Search for a unique string near the content you want to change. Copy the exact text including indentation. Paste it verbatim under Replace:. Do not reconstruct from Twine source -- the HTML encoding is the ground truth.

Passage content vs script block content

Twine passage content is HTML-encoded in the game file: `<<if $x>>` becomes `&lt;&lt;if $x&gt;&gt;` in the HTML. JavaScript in the `<script>` block is NOT HTML-encoded -- it stores raw characters. This distinction matters critically for Replace: anchors.

NOTE: v0.7.4 fixes a long-standing bug where Replace: anchors targeting JS object entries in the script block (setup.storyhints.db, setup.inclinations, etc.) would silently fail. v0.7.4 adds a two-pass search: encoded first, then raw if encoded misses and no passage tag is found in the anchor.

The [to] marker

Use [to] inside a Replace: block to automatically capture everything between a prefix and suffix. The patcher fills in whatever is between them and uses the complete result as the search target.

```
Replace:
<<set $gangbangChance to [to]>>
With:
<<set $gangbangChance to 0.1>>
```

NOTE: Only one [to] per Replace: block. If no match is found the block is skipped gracefully.

ReplaceReg [pattern]:

Uses a Python regex pattern to find a match in the game HTML, then replaces one or more capture groups. Useful when surrounding context is variable but the pattern is stable.

```
ReplaceReg [(?s)somePrefix(GROUP_CONTENT)someSuffix]:
With{1}:
replacement for group 1
```

NOTE: Use (?s) to enable DOTALL mode (. matches newlines). Group 0 is the full match. Multiple With{N}: blocks in one directive replace different groups. Applied right-to-left.

6. Structural Directives

The structural directives anchor on names rather than content lines, giving them near-Tier S resilience. They survive most game updates unless the game renames the specific array, class, or object they target.

```
Append Array [setup.hairLengths]:
"Very Long",
"Extremely Long",

Append Variable to Class [Person]:
this.pregnancyWeek = 0;

Append Function to Class [Person]:
getPregnancyStage() {
  if (this.pregnancyWeek < 13) return "first";
  return "second";
}

Insert Into Array [setup.kinklist]:
"newKink",

Insert Into Object [setup.config]:
myModKey: true,
```

NOTE: All structural directives support optional In Function scoping: Append Array [someArray In Function [setup.init]]:

Append Array vs Insert Into Array

Both append content to a named array. Append Array inserts content before the array's closing bracket. Insert Into Array also inserts at the end. In practice both produce the same result for simple value arrays.

Append Variable to Class: ES6 vs prototype

The patcher detects whether the target class uses ES6 class syntax or prototype-style constructor syntax and inserts variables in the correct location.

```
// ES6 class:
Append Variable to Class [Person]:
this.myModProp = 0;
// Inserts inside constructor() {}

// Prototype style:
Append Variable to Class [Person]:
this.myModProp = 0;
// Inserts at end of Person = function(...) { ... }
```

Insert Into Object is ideal for script-block JS objects

When adding entries to named JS objects in the `<script>` block (like `setup.storyhints.db` or `setup.inclinations`), always prefer Insert Into Object over Replace:. It anchors on the object name and is immune to the script-block encoding issue that Replace: had before v0.7.4.

BEST PRACTICE: For any named object in the game script block, use Insert Into Object. It is Tier S. Replace: on script-block content is Tier 5 even after the v0.7.4 two-pass fix.

7. Guard Clauses

Any directive block can include guard lines that make the patch conditional. Guards are evaluated against the current HTML at patch time. If any guard fails, the entire block is skipped and logged.

Guard	Condition
IfExists [value]:	Apply only if string exists anywhere in the HTML
IfNotExists [value]:	Apply only if string is absent from the HTML
IfRegExists [pattern]:	Apply only if regex matches the HTML
IfRegNotExists [pattern]:	Apply only if regex does not match the HTML
IfModLoaded [filename]:	Apply only if the named .mod file is loaded
IfModNotLoaded [filename]:	Apply only if the named .mod file is NOT loaded
IfPassageExists [name]:	Apply only if passage exists in HTML or registry

IfPassageNotExists [name]:	Apply only if passage does not exist
IfFunctionExists [name]:	Apply only if function exists in script block
IfFunctionNotExists [name]:	Apply only if function does not exist
IfPassageContains [Name text]: (v0.7.0)	Apply only if named passage body contains the text (pipe-separated, decoded Twine)
IfPassageHasTag [Name tag]: (v0.7.0)	Apply only if named passage has the exact tag token (pipe-separated)

```

Replace:
<<set $gangbangChance to 0.05>>
With:
IfExists [setup.Events.db]:
IfNotExists [gangbangEnabled]:
IfRegExists [setup\.people\.generate\s*=\s*function]:
<<set $gangbangChance to 0.1>>

```

Guards are placed inside the With: body. Guards preceding a directive on a separate line are automatically hoisted into the correct body region. All guards of the same type must pass. Guards are stripped from the With: body before injection -- they never appear in the patched HTML.

Guards in Depth

IfPassageContains [PassageName|text]:

Fires only if the named passage body contains the search text after HTML-decoding. The PassageName and text are separated by a pipe character. Use this when the same anchor string appears in multiple passages and you need to condition on the specific content of one.

```

Append To Passage [EventShower]:
With:
IfPassageContains [EventShower|setup.soaked_level]:
// Only appends if the soaked mod has already modified this passage
<<run setup.cm_soaked_shower_bonus()>>

```

IfPassageHasTag [PassageName|tag]:

Fires only if the named passage has the exact tag token. More reliable than IfExists [tags="...newtag..."] patterns which can match partial tag names.

```

Append To Passage [MyEvent]:
With:
IfPassageHasTag [MyEvent|nobr]:
// Only appends if this event passage has the nobr tag
<<run setup.cm_mymod_event()>>

```

IfModLoaded vs IfModNotLoaded

IfModLoaded [filename]: fires only if the named .mod file is present in the mods/ folder. Use for optional cross-mod hooks in submods. Main mods should use typeof checks in the body instead, for robustness.

```

Append To Passage [EventCleanup]:
With:
IfModLoaded [CM_BCMod.v2]:
// Only appends if the BC mod is loaded

```

```
<<run setup.cm_bc_cleanup_hook()>>
```

NOTE: IfModLoaded guards are ideal for optional cross-mod hooks in submods. Main mods should use typeof checks in the body (if (typeof setup.cm_bc_check !== "undefined")) for robustness in case the mod is loaded out of order.

8. Processing Order

During parsing, directives are sorted into separate lists. During application, they execute in a fixed seven-phase order.

Phase	What runs	Details
0 (v0.7.0)	Dep graph sort	If cm_dependencies.json is present, mods are topologically sorted. Falls back to Priority:/filesystem order on missing file or cycle.
1	Replace: entries	Replace:/With: pairs in mod load order. v0.7.4: two-pass anchor search. Passage-targeting entries use name-based fallback if exact match fails.
2	ReplaceReg entries	Regex-based replacements against the full HTML.
3	Structural entries	Append Array, Append Variable/Function to Class, Insert Into Array/Object.
4	Func/passage-scope d (sorted)	Add Function (sorted first), then Insert Into Function, Replace Function, Replace Function Signature, IA/IB, Delete Block/Span, RIP/RIF, DIP, Move/Rename, Tag ops, Prepend/Append To Passage, Add Variable.
5	Hook entries	Function wrapping with before/after/around hooks. v0.7.4: conflict fix.
6	Failed Replace: sweep	Retry failed Phase 1 entries against passage registry.
7	Registry injection	(a) Add JS to script block, (b) Add CSS to style block, (c) Add Events to Events.db, (d) Add Passage before storydata close. PID renormalization runs after.

WARNING: Add Passage / Add Javascript / Add CSS / Add Events are injected in Phase 7 after all other directives have run. Replace Function (Phase 4) cannot target a function added by Add Function in the same load. Add Function IS sorted before Insert Into Function within Phase 4.

9. How Multiple Mods Interact

All .mod files in the mods/ folder are loaded and applied on every patch run. The patcher restores from backup first, so running it twice does not double-apply. v0.7.0 load order is determined by cm_dependencies.json; otherwise alphabetical filename order.

Load order in v0.7.0+

If `cm_dependencies.json` is present, mods are loaded in topological sort order (parents before children). Within the same dependency level, mods are sorted by the `Priority:` field in their structured header (default 1000, lower = earlier). Without `cm_dependencies.json`, mods load in filesystem order (typically alphabetical).

Phase ordering governs injection order

Two mods that both use `Insert After` to inject near the same anchor will both succeed -- each finds the anchor text (not consumed by the first injection) and injects in mod load order. Two mods that both use `Replace:` on the same anchor will conflict -- only the last applied wins, and the conflict is logged.

Add Function vs Replace Function

Use `Add Function` for new functions your mod introduces -- version-safe, never overwrites existing code. Use `Replace Function` only to modify existing game functions -- not version-safe, will need updating if the game changes the function body.

Two mods touching the same function

If two mods both need to modify the same function, use `Hook + Add Function` in each mod. Each hook runs independently without overwriting the other, and both coexist. v0.7.4 fixes the previous bug where the second mod's hook was silently dropped -- all mods now fire in load order.

```
// ModA: adds tracker
Add Function [setup.modA_track]: ...
Hook [setup.build_encounter]:
after:
    setup.modA_track();

// ModB: adds its own hook
Add Function [setup.modB_track]: ...
Hook [setup.build_encounter]:
after:
    setup.modB_track();

// v0.7.4: both fire -- modB hook body is appended into modA's wrapper.
// Before v0.7.4: only modA fired; modB hook was silently dropped.
```

10. Structured Mod Header

In v0.7.0, the patcher parses specific fields from the mod file header (`#` comment lines at the top of the file). These fields are optional but strongly recommended for any mod distributed to other users.

Field	Format	Purpose
Mod Name:	# Mod Name: CM_MyMod	Display name for logs and toolchain output
Version:	# Version: 1.0.0	Semantic version string for the mod
Author:	# Author: CoderMaverick	Mod author credit (used in release toolchain)

Requires:	# Requires: CM_ModA.v1, CM_ModB.v1	Advisory only -- emits warning if absent. Does not enforce ordering.
Priority:	# Priority: 500	Integer. Lower = loads earlier within same dep level. Default 1000.
Description:	# Description: What this mod does	One-line description for logs and toolchain output

NOTE: The Requires: field does NOT enforce load ordering -- only cm_dependencies.json does. Requires: emits a warning to MainPatchLog.txt if a named mod file is not found in the mods folder, but does not block patching.

Complete mod header example

```
# Mod Name: CM_MyMod
# Version: 1.3.0
# Author: CoderMaverick
# Requires: CM_ParentMod.v1, CM_AnotherMod.v2
# Priority: 800
# Description: Expands the gangbang system with new NPC archetypes.
#
# v1.3.0: Added 3 new NPC archetypes. Fixed anchor drift on EventGB.
# v1.2.1: Patched for 7.9c anchor changes in EventShower.
# v1.2.0: Added BC integration via cm_hc_gb_aftermath_check.
# v1.1.0: Initial release.
```

```
Add Function [setup.cm_mymod_check]:
setup.cm_mymod_check = function(config) {
    return config && config.people && config.people.length >= 3;
};
```

All header fields are optional. Unrecognized # lines in the header are treated as freeform comments and ignored. Fields are only parsed until the first non-# line (the first directive). Fields after the first directive are ignored.

11. Escaping Rules in Depth

Understanding the escaping context for every part of a .mod file is the single most important thing for avoiding silent failures.

What gets HTML-encoded in the game file

Twine stores passage content in the HTML as HTML-encoded text. The string `<<if $foo>>` in Twine source becomes `&lt;&lt;if $foo&gt;&gt;` in the HTML file. This encoding only applies to passage content -- not to the `<script>` block.

What is NOT HTML-encoded

JavaScript in the game's `<script>` block is NOT HTML-encoded. The string `<<if>>` does not appear in the script block. When your Replace: target is inside a `<script>` block, write the JS syntax directly without HTML encoding.

NOTE: v0.7.4 fixes a bug where Replace: anchors targeting script-block content would silently fail because the patcher was HTML-encoding the anchor before searching. v0.7.4 adds a two-pass search: encoded first, then raw anchor if miss and no passage tag found in the anchor string.

The patcher auto-escape (v0.7.0)

In v0.7.0, With: bodies for passage content directives are automatically escaped before injection. Write raw Twine in your With: body and the patcher converts it to the HTML-encoded form. The <e> tag is no longer needed for new mods.

Escaping quick reference

Location	What to write
Replace: block	HTML-escaped form: copy from game HTML directly
With: body (Twine passage)	Raw Twine in v0.7.0+ (auto-escaped); use <e>...</e> for v0.6.0 compat
With: body (JS script)	Write raw JS -- patcher unescapes automatically for script context
Add Passage: body	Write raw Twine -- patcher auto-escapes passage bodies
Add Javascript: body	Write raw JS -- no escaping needed
Add Function: body	Write raw JS -- no escaping needed
Insert After anchor (passage)	Raw Twine -- patcher encodes before searching passage content
Insert After anchor (script block)	Raw JS -- script block is not HTML-encoded

12. KittyValidate in Depth

KittyValidate.py checks every directive anchor against the live game HTML without patching anything. Run it before applying a mod to catch anchors that will fail.

```
python KittyValidate.py CourseOfTemptation.html MyMod.mod
python KittyValidate.py --verbose CourseOfTemptation.html mods/
python KittyValidate.py --write-report CourseOfTemptation.html MyMod.mod
```

Status	Meaning
safe	Anchor found exactly once -- will apply correctly
ambiguous	Anchor found 2+ times -- matches first occurrence only; verify location
will fail	Anchor not found -- directive will error at patch time
would be added	Add Passage / Add Function target does not exist yet (correct)
already exists -- will be skipped	Add Passage / Add Function target already present

ST_UNSCOPED (v0.7.0)	Global anchor found but Insert Into Function would be safer
----------------------	---

WARNING: The "ambiguous" status does not stop patching -- it is advisory. But an ambiguous Insert After that matches the wrong passage will inject code in the wrong location. Always resolve ambiguous before shipping a mod.

Zero-failure policy

All directive anchors must report "safe" or "would be added" before a mod is released. Any "will fail" or "ambiguous" status requires authoring changes.

ST_UNSCOPED in depth

KittyValidate reports ST_UNSCOPED when an Insert After anchor exists in the HTML but the same string appears in more than one function scope. Upgrade to Insert Into Function to scope the injection correctly.

```
// ST_UNSCOPED flagged:
Insert After [var result = {}]:
With:
result.modFlag = true;

// Fix: scope with Insert Into Function
Insert Into Function [setup.build_encounter]:
var result = {};
With:
result.modFlag = true;
```

13. KittyDiffAssist (Modder Edition)

KittyDiffAssist.py generates detailed failure reports when Replace: anchors fail after a game update. For each failed anchor, it produces:

- A per-failure diff report (DiffAssist_<mod>.txt) showing what the anchor was, what it found nearby in the new HTML, and the surrounding context diff.

- An auto-ported draft mod file (PortDraft_<mod>.mod) with best-guess updated anchors based on fuzzy matching against the new HTML.

- DiffAssist does not catch Tier 3 (Replace Function) silent drift -- when the function name still exists but the body has changed, it reports the anchor as safe. Manual comparison is required for Replace Function directives.

```
python KittyDiffAssist.py new_game.html MyMod.mod
python KittyDiffAssist.py --all new_game.html mods/
```

Output files

```
logs/DiffAssist_MyMod.txt    <- per-failure diff report
logs/PortDraft_MyMod.mod    <- auto-updated anchor draft
```

NOTE: PortDraft is a starting point, not a finished port. Always validate PortDraft against the new HTML with KittyValidate before using.

14. KittyPort

KittyPort.py is an automated porting tool that compares old and new game HTML versions, identifies which Replace: anchors have drifted, and generates updated anchor candidates. It classifies the result: ported (no failures), review (failures found, PortDraft generated), or preexisting-fail (anchor was also missing from old HTML).

```
python KittyPort.py old_game.html new_game.html mods/  
python KittyPort.py --upgrade old_game.html new_game.html mods/
```

17. Debugging Failures

Silent failures

The most common failure mode: a Replace: block silently does not apply because the search text does not match exactly. Always check ModPatchLog.txt for "No match found" entries after every patch run. KittyValidate catches these before patching.

Encoding mismatch

If the Replace: anchor does not match, the most common cause is encoding mismatch. Check whether the anchor is in a passage body (HTML-encoded) or in the script block (raw JS). v0.7.4 fixes the most common script-block encoding issue automatically with two-pass search, but passage-body encoding issues still require copying the anchor directly from the game HTML.

Comments between directives

A comment line between the end of one With: body and the next directive, with a blank line before the comment, causes the comment to be absorbed into the preceding With: body and injected as literal content into the game.

```
// WRONG -- comment gets injected:  
Append To Passage [StoryInit]:  
With:  
<<set $myVar to 1>>  
  
// This comment gets injected as visible text!  
Insert After [someAnchor]:  
  
// CORRECT:  
Append To Passage [StoryInit]:  
With:  
<<set $myVar to 1>>  
Insert After [someAnchor]:
```

Getting the exact search text

Open the game HTML in a text editor. Search for a unique string near the content you want to change. Copy the exact text including indentation. Paste it verbatim under Replace:. Do not reconstruct from Twine source -- the HTML encoding is the ground truth.

18. Porting After Game Updates

When a new version of the game releases, run KittyValidate against the new HTML before patching. Any "will fail" entries indicate anchors that have drifted.

Porting cost by tier

Tier	Directives	Fails when
S	Add Passage, Add JS/CSS/Events, Add Function, Add Variable, Rename, Move, Tag ops	Never
1	Prepend/Append To Passage	Passage renamed or removed
2	Hook, Insert Before, Insert After	Function/anchor renamed or removed
2.5	Insert Into Function, Replace In Function, Replace Func Sig, Delete In Passage	Name or anchor within body changes
3	Replace Function	Function renamed; body drift is silent (you own the full body)
4	Replace In Passage, Delete Span	Passage renamed or anchor changes
5	ReplaceReg, Replace:	Any targeted line changes
S	Append Array/Var/Func, Insert Into Array/Object, Delete Block	Name renamed or removed

The invisible Tier 3 cost

Replace Function carries a cost beyond the anchor -- you silently own the full function body. If the game developer adds a bug fix, a new parameter, or a new behavior to that function in a future update, your mod discards it without any log entry. DiffAssist does not catch this because the Replace Function anchor (function name) still matches. You must manually compare your replacement body against the new game version's body.

Porting from v0.6.0 to v0.7.0

Change	Action required	Impact
kitty_config.ini -> kitty_config.toml	Auto-migrated on first run. No action.	None
cm_dependencies.json	Optional. Add if you need load ordering guarantees.	None if absent
Hook around: timing	New feature. No existing mods affected.	None

IfPassageContains IfPassageHasTag	/	New feature. No existing mods affected.	None
With: auto-escape		<e> tags still work. No action needed.	None
BeautifulSoup 4		Internal only. No mod changes needed.	None
ST_UNSCOPED warning in KittyValidate		Advisory only. Consider scoping global IA anchors.	No breakage; advisory upgrade

18b. v0.7.0 Changes Quick Summary for Modders

Feature	What changed	What you do
Dep graph	Phase 0 sorts mods topologically if cm_dependencies.json present	Add cm_dependencies.json if load order matters for your mods
Hook around:	New timing -- full control over _result, must explicitly return	Use for intercepting and transforming return values
IfPassageContains	New guard: IfPassageContains [Name text]: -- pipe-separated, decoded Twine	Use when conditioning on passage content not just existence
IfPassageHasTag	New guard: IfPassageHasTag [Name tag]: -- pipe-separated	Use instead of fragile IfExists [tags="...newtag..."] patterns
With: auto-escape	With: bodies now auto-escaped -- write raw Twine directly	Existing <e> tags still work; new mods can omit them
Structured header	Mod Name:, Version:, Author:, Requires:, Priority: parsed	Add Priority: to control load order within dependency level
Patch cache	Cached results in mods/logs/cache/	No action; automatic. Delete cache/ to force full re-patch
ST_UNSCOPED	KittyValidate warns when IA anchor would be safer as IIF	Upgrade Insert After to Insert Into Function where flagged

18c. v0.7.4 Changes: Modder Summary

Feature	What changed	What you do
Patch cache	Content-addressed SHA-256 cache. ~4 min to ~0.2 s on hit.	Disable during active development: [cache] enabled = false

Batch pre-pass	Append To Passage and Insert After/Before grouped by target.	No action. Transparent to mod authors. Improves startup time.
Two-pass Replace: search	Replace: now finds script-block targets correctly via raw fallback.	No mod changes needed. Write anchors the same way you always have.
Auto-close	Terminal exits after browser launch. No "Press Enter" prompt.	No action. Watch mode and errors unaffected.
Hook conflict fix	Second+ mod hooks on same function now append, not silently drop.	No action. Cross-mod Hook conflicts now correctly fire all bodies.
Cache on failure fix	Cache not written when patch has failures.	No action. Automatic behavior fix.
Cache key fix	Log files excluded from cache key hash.	No action. Eliminates spurious cache misses from log file changes.

19. v0.7.4 Patch Cache

The patch cache dramatically speeds up the authoring loop. On a cache hit, the full patch phase is skipped and the cached result is used directly. The cache key covers the vanilla HTML and every .mod file -- any change to either invalidates it automatically.

During active development -- disable the cache

While iterating on a mod, disable the cache so every run reflects the current state of your .mod file:

```
[cache]
enabled = false    # recommended during active mod authoring
```

Re-enable it after you reach a stable point. The cache is most useful for end users who are not actively developing mods -- they benefit from the fast startup on every session.

NOTE: The cache is NOT written when the patch run has failures. Only clean patch runs are cached. A run with any failed directives always produces a fresh patch on the next run.

Cache directory and pruning

The cache is stored in cache/ alongside the patcher. It is automatically pruned to max_entries entries (default 10), removing oldest first. Delete the cache/ folder manually to force a full re-patch without disabling the cache setting.

```
[cache]
enabled = true
max_entries = 10    # prune oldest entries beyond this count
```

Why log files are excluded from the cache key

Before v0.7.4, the cache key accidentally included log files in the mods/ directory. Any log file change (even from reading it in a text editor) would invalidate the cache and force a full re-patch. v0.7.4 fixes the key to cover only .mod files and the vanilla HTML.

20. v0.7.4 Replace: Two-Pass Anchor Search

Background: the encoding problem

Before v0.7.4, Replace: anchors were HTML-encoded before searching. This is correct for passage content (which is stored HTML-encoded in the game file), but wrong for JavaScript object entries in the `<script>` block. Script block content stores raw characters -- raw quote characters, not `"` entities.

The result was that Replace: anchors targeting JS object entries in the script block would silently fail with "No match found" in the log, even though the anchor text was written exactly correctly.

What changed in v0.7.4

The patcher now tries the encoded anchor first (correct for passage content). If the encoded search misses and the anchor contains no passage tag (indicating a script-block target rather than a passage target), the raw unencoded anchor is tried as a fallback. The log distinguishes fallback hits:

```
Replace: encoded anchor missed -- raw script-block fallback succeeded
for 'setup.storyhints.db entry...' (v0.7.4)
```

What you need to do

Nothing. Write Replace: anchors targeting script-block content the same way you always have -- raw JS syntax, no HTML encoding. The patcher now finds them correctly on the first attempt.

What you SHOULD do instead

For any named JS object entry in the script block, prefer Insert Into Object over Replace:. This has always been the recommendation and it is now more important to be aware of, since the silent failure masked the issue before.

```
// FRAGILE (Replace: on script-block entry -- Tier 5):
Replace:
"We All Cheer For Cheerleaders": {
  tags: ["class"],
},
With:
"We All Cheer For Cheerleaders": {
  tags: ["class"],
  myModProp: true,
},

// RESILIENT (Insert Into Object -- Tier 5):
Insert Into Object [setup.storyhints.db]:
"My New Story Hint": {
  tags: ["class"],
  text: "Something interesting happened.",
},
```

21. v0.7.4 Hook Conflict Fix and Duplicate Hook Rule

Cross-mod Hook conflicts (fixed in v0.7.4)

Before v0.7.4, when two mods both hooked the same function, the second hook was silently dropped. The conflict was logged, but only the first mod's hook body executed.

v0.7.4 fix: when a second (or third, or fourth) mod hooks the same function, the new hook body is appended into the existing wrapper. All mods fire in load order. The original function is called once. No hook body is lost.

```
// ModA.mod:
Hook [setup.build_encounter]:
after:
  setup.modA_track();

// ModB.mod:
Hook [setup.build_encounter]:
after:
  setup.modB_track();

// v0.7.4 generated wrapper (both fire):
setup.build_encounter = function() {
  var _result = setup_build_encounter__orig.apply(this, arguments);
  setup.modA_track();    // from ModA (load order first)
  setup.modB_track();    // from ModB (load order second)
  return _result;
};
```

NOTE: This fix applies to cross-mod Hook conflicts. For same-mod duplicate Hooks on the same function with the same timing, see the next section.

Same-mod duplicate Hook directives: a functional bug

When the same mod has two Hook [setup.func]: around: directives (same function, same timing), the patcher v0.7.4 cross-mod handling does NOT apply. The second around: hook wraps the first hook's wrapper. The second call to _original() inside the second hook body calls the first hook's wrapper, not the real function -- effectively double-applying the first hook's logic.

```
// BUG: two around: hooks on the same function in one mod
Hook [setup.sport_npc_skill]:
around:
  _result = _original.call(this, _arg0, _arg1);
  // cycle modifier logic
  return _result;

Hook [setup.sport_npc_skill]:    // DUPLICATE around: -- BUG
around:
  _result = _original.call(this, _arg0, _arg1);
  // pregnancy dropout logic
  return _result;

// CORRECT: one merged around: hook body
```

```
Hook [setup.sport_npc_skill]:
around:
    _result = _original.call(this, _arg0, _arg1);
    // cycle modifier logic
    // pregnancy dropout logic
    return _result;
```

WARNING: Duplicate same-timing Hook directives within one mod file double-apply logic. Different-timing hooks (around: and after:) on the same function from the same mod are not duplicates -- they serve different purposes and may remain separate.

22. Surgical Updates: Extend, Never Duplicate

When updating an existing mod to add new content to a passage or function it already touches, always extend the existing directive rather than adding a second one. This is the primary source of same-mod duplicate conflicts and must be prevented at authoring time.

```
// WRONG: v1.2.0 adds a second Append To Passage for the same target
// (v1.1.0 block already in the file):
Append To Passage [CMPharmacy]:
With:
<<existing content from v1.1.0>>

// v1.2.0 update (WRONG -- creates a duplicate):
Append To Passage [CMPharmacy]:
With:
<<new content from v1.2.0>>

// CORRECT: extend the existing block
Append To Passage [CMPharmacy]:
With:
<<existing content from v1.1.0>>
<<new content from v1.2.0>>
```

The same rule applies to Hooks. If the mod already has Hook [setup.func]: after: and an update needs more logic in that hook, add it to the existing hook body -- not a new Hook directive.

BEST PRACTICE: Before adding any directive in a mod update, check whether the mod already has a directive for that target. Extend the existing With: body or hook body. Never create a second directive for the same target.

Append To Passage duplicates

Multiple Append To Passage [X]: blocks in one mod file produce conflict log entries but do not break functionality -- all With: bodies are applied sequentially. They should still be consolidated into a single directive for cleanliness and to eliminate the noise.

```
// WRONG: two blocks for the same target
Append To Passage [EventCleanup]:
With:
<<run setup.mymod_cleanup()>>

Append To Passage [EventCleanup]:    // duplicate
```

```

With:
<<set V._cmModFlag to false>>

// CORRECT: one consolidated block
Append To Passage [EventCleanup]:
With:
<<run setup.mymod_cleanup()>>
<<set V._cmModFlag to false>>

```

23. Orphan Directives

An orphan directive is one that cannot fire correctly because its target does not exist. Common cases:

- An Append To Passage targeting a passage no mod or vanilla game defines.
- A Hook targeting a function not defined anywhere in the script block.
- An Add Events: entry whose passage field names a passage that does not exist.
- An Insert Into Function targeting a function nobody defines.
- A Replace In Passage targeting a passage name with a typo.

Orphan directives for hook surfaces (passages or functions that a parent mod provides for submods to extend) are valid when documented in the mod header's Provides: or comment block. Undocumented orphans that can never fire are dead code.

NOTE: KittyValidate reports "will fail" for directives whose anchor is absent from the HTML. Run KittyValidate against every game version after a game update to find newly orphaned directives.

Hook surface passages

A "hook surface" is a passage or function that a mod adds specifically so that submods can extend it. For example, a main mod might add a passage EventCMMMyModCleanup and document in its header that submods may Append To Passage [EventCMMMyModCleanup] to register their own cleanup logic.

Submods that do this will have orphan directives against any game HTML that does not include the parent mod. This is expected and valid -- but it must be guarded with IfPassageExists or IfModLoaded so the directive fails gracefully rather than erroring when the parent mod is absent.

```

// In the submod:
Append To Passage [EventCMMMyModCleanup]:
With:
IfModLoaded [CM_MyMod.v1]:
IfPassageExists [EventCMMMyModCleanup]:
<<run setup.cm_submod_cleanup()>>

```

24. Delete Directives in Depth

Delete Block [name]:

Removes an entire named passage from the game HTML including its opening tag, all content, and closing </tw-passagedata>. No duplicate check -- if the passage is already absent, the directive logs a failure but does not abort patching.

```
Delete Block [SomeUnwantedPassage]:
```

WARNING: Use Delete Block sparingly. If the dev keeps the passage but your mod deletes it, any game logic that depends on it will break silently. Always verify there are no remaining references to the passage name in the vanilla game before using this.

Delete Span [start] To [end]:

Deletes everything from start_anchor through end_anchor, inclusive of both anchors. No With: body is needed. Both anchors must be present in the HTML and start must precede end. Preferable to Replace: for pure deletion because there is no With: body to maintain.

```
Delete Span [<<set _oldvar to true>>] To [<<unset _oldvar>>]:
```

Delete In Passage [name]:

Scoped deletion that removes anchor text from the named passage body only. No With: body needed. Write raw Twine in the anchor -- the patcher encodes it. If the passage is not found or the anchor is not found within the passage body, the block fails and is logged.

```
Delete In Passage [ShowerStall]:  
<<set _oldvar to true>>
```

Multiple Delete In Passage blocks with different anchors on the same passage are valid -- they are sequential deletions, not duplicates. This is the one case where multiple same-target directives are correct.

25. Common Authoring Mistakes

Anchors that are too short or too common

A Replace: anchor like <<set \$count to 0>> will match the FIRST occurrence of that exact string anywhere in the 16MB+ game HTML. If that string appears in 12 different passages, your mod will always patch the same first one. Keep anchors unique: use at least 2-3 lines of surrounding context, or switch to Replace In Passage to scope the match.

Replace Function when Hook would do

Replace Function gives you the entire function body to own and maintain. If the game developer fixes a bug or adds a parameter in the next update, your mod silently discards those changes. Reserve Replace Function for cases where you genuinely need to restructure the entire function. For simple additions, use Hook + Add Function instead.

Not guarding Add Javascript: setup code

Add Javascript: has no duplicate check. If your setup code runs twice (because the patcher was run twice, or because both a parent and child mod include the same setup block), the second execution can corrupt game state.

```
// WRONG: runs on every page load, including duplicated from re-patch:  
Add Javascript:  
setup.myConfig = { enabled: true };
```

```
// CORRECT: guard against re-initialization:
Add Javascript:
if (!setup.myConfig) {
  setup.myConfig = { enabled: true };
}
```

Writing both formats for the same passage

Mixing shorthand and full `<tw-passagedata>` format in the same mod for the same passage name will result in the second being silently skipped. Use one format consistently per passage.

Inner replacements and nested conflicts

If a `Replace:` target only exists inside another mod's `With:` body (not in the base HTML), the patcher detects this and applies the inner replacement directly into the host block. This is handled transparently. But if two mods both perform inner replacements on the same host block, the order of application determines which one wins.

26. v0.7.4 New Features Summary for Modders

Feature	Impact on mods	Action required
Patch cache	Startup time drops from ~4 min to ~0.2 s on cache hit.	Disable during active authoring: [cache] enabled = false
Batch pre-pass	Append To Passage and Insert After/Before run faster on large mod sets.	None. Transparent.
Two-pass Replace: search	Replace: on JS script-block targets (setup.X.db entries etc.) no longer silently fails.	None. Write anchors the same way. Prefer Insert Into Object for new script-block additions.
Auto-close	Terminal exits after game opens. No "Press Enter" prompt.	None.
Hook conflict fix	Second+ mod hooks on the same function now fire. Previously only the first mod fired.	None. If you worked around this by merging hooks from different mods into one mod, you can now split them back into their natural separate files.
Cache on failure fix	Cache not written when patch has failures.	None. Prevents stale cached patched HTML from persisting after a failed patch run.
Same-mod duplicate Hook rule	Same-timing Hook duplicates in one mod file double-apply logic (this was always a bug).	Audit your mod files for duplicate same-timing Hook directives. Merge them.

28. Practical Walkthrough: Building a Cross-Mod Submod

This walkthrough demonstrates the complete authoring process for a submod that integrates two parent mods. The example is a hypothetical CM_BC_FS.v1 -- a submod that connects the BioCycle system (CM_BC) to the Fans system (CM_FS). The same patterns apply to any two-parent submod.

Step 1: Define what the submod needs to do

Before writing any directives, write out in plain language what the submod adds. For CM_BC_FS: when the PC is in the fertile window, fan encounter heat is increased; when a fan NPC is at obsessive tier, they notice the PC's fertile window and comment; fan sugar daddy NPCs get a breeding-kink variant at peak fertility. Each of these maps to a different kind of directive.

Step 2: Check parent mod detection anchors

Every directive in the submod must be guarded by both parent mods. Look up the detection anchor each parent mod documents in its header Provides: block:

```
# CM_BC detects via:    IfPassageExists [CMCreampieAftermath]
# CM_FS detects via:    IfFunctionExists [setup.cm_fans_count]

# EVERY directive in the submod gets both guards:
Append To Passage [SomeTarget]:
With:
IfPassageExists [CMCreampieAftermath]:
IfFunctionExists [setup.cm_fans_count]:
<<run setup.cm_bc_fs_fan_heat_tick()>>
```

Step 3: Write your logic in Add Function first

All substantive JS logic goes in Add Function blocks. These are Tier S -- they never need porting. The passage hooks (Tier 1-2) just call them.

```
Add Function [setup.cm_bc_fs_fan_heat_mod]:
setup.cm_bc_fs_fan_heat_mod = function(npc) {
  if (typeof setup.cm_hc_pc_is_fertile !== "function") return 0;
  if (typeof setup.cm_fans_tier !== "function") return 0;
  var fertile = setup.cm_hc_pc_is_fertile();
  var tier = setup.cm_fans_tier(npc);
  if (!fertile) return 0;
  if (tier === "Obsessive" || tier === "Stalker" || tier === "Extreme") return 15;
  if (tier === "Dedicated") return 8;
  return 3;
};
```

NOTE: The function guards every cm_hc_* and cm_fans_* call with typeof checks. Even though the guards on the directive guarantee both mods are loaded, typeof guards inside the function make it safe to call from anywhere.

Step 4: Hook into the parent systems

Now write the smallest possible directive that calls the function. Use Append To Passage for passage hooks, Hook for function hooks.

```
# Add heat modifier to fan encounter event
Append To Passage [EventCMFSObsessiveFanEncounter]:
With:
IfPassageExists [CMCreampieAftermath]:
IfPassageExists [EventCMFSObsessiveFanEncounter]:
<<if typeof setup.cm_bc_fs_fan_heat_mod === "function">>
<<run setup.cm_fans_heat_add($eventnpc, setup.cm_bc_fs_fan_heat_mod($eventnpc))>>
<</if>>
```

Step 5: Add the passage content

New passages for submod-specific scenes go in Add Passage blocks. Always give them a clearly namespaced name -- CM_BC_FS_* for this submod.

```
Add Passage: EventCMBCFSFertileFanBeat
tags: noevents nobr
<<if typeof setup.cm_bc_fs_fan_heat_mod !== "function">><<return>><</if>>
<<if !setup.cm_hc_pc_is_fertile()>><<return>><</if>>
The air between you carries a charge today...
```

Step 6: Register events only when both parents are present

Add Events: entries must be guarded identically to the directives that created the passage they reference. Use IfPassageExists on your own passage as the guard -- if the Add Passage fired, the passage exists.

```
Add Events:
{
  passage: "EventCMBCFSFertileFanBeat",
  tags: ["campus", "dorm"],
  chance: 0.03,
  conditions: [],
},
IfPassageExists [CMCreampieAftermath]:
IfFunctionExists [setup.cm_fans_count]:
```

29. Cross-Mod Integration Patterns

There are two ways to make a directive conditional on another mod being loaded: IfModLoaded guards and typeof checks. They serve different purposes and should not be confused.

IfModLoaded guards (on the directive)

IfModLoaded [ModFile.v1]: prevents the entire directive from firing if the named .mod file is absent. Use this in submods where the entire directive is useless without the parent mod. The guard fires at patch time.

```
Append To Passage [EventCleanup]:
With:
IfModLoaded [CM_BC.v2]:
<<run setup.cm_bc_cleanup()>>
```

typeof checks (inside the body)

typeof checks are runtime guards. They fire when the game actually runs the passage. Use these inside Add Function bodies, passage bodies, and event passages to safely call functions from optional mods.

```
Add Function [setup.mymod_do_thing]:
setup.mymod_do_thing = function() {
  // Always safe to call -- guards inline
  if (typeof setup.cm_fame_add === "function") {
    setup.cm_fame_add("social", 5);
  }
  if (typeof setup.cm_hc_pc_is_fertile === "function") {
    var fertile = setup.cm_hc_pc_is_fertile();
  }
};
```

Which to use when

Main mods use typeof checks inside function bodies for all optional integrations. Submods use IfModLoaded guards on directives (at patch time) and typeof checks inside function bodies (at runtime). Using typeof only (no IfModLoaded) is safe but produces log entries about skipped directives when the optional mod is absent, which is noise. Using both is the cleanest approach.

NOTE: IfPassageExists and IfFunctionExists are preferred over IfModLoaded when the detection target is a specific passage or function rather than a filename. They are more robust if the mod is ever renamed.

30. Namespace Conventions and Save Compatibility

State namespaces

All CM mods store state in V._ prefixed objects with a unique mod prefix. This prevents collisions between mods and makes save state auditable.

```
// State namespace pattern:
V._cmMyMod = V._cmMyMod ?? {};
V._cmMyMod.level = V._cmMyMod.level ?? 0;
V._cmMyMod.lastEvent = V._cmMyMod.lastEvent ?? null;

// Per-NPC state pattern:
V._cmMyMod[npcName] = V._cmMyMod[npcName] ?? {};
V._cmMyMod[npcName].heat = V._cmMyMod[npcName].heat ?? 0;
```

The ?? default pattern

Every state variable must be initialized with ?? (nullish coalescing) rather than simple assignment. This ensures old saves (created before your mod was installed) load without errors. The variable exists as undefined in old saves -- ?? replaces undefined with the default value without overwriting existing state.

Lazy initialization in every function

Every function that reads mod state must also initialize it lazily at the top. This handles saves where StoryInit has already run but the mod was installed later.

```
Add Function [setup.cm_mymod_init]:
setup.cm_mymod_init = function() {
  V._cmMyMod = V._cmMyMod ?? {};
  V._cmMyMod.level = V._cmMyMod.level ?? 0;
  V._cmMyMod.lastEvent = V._cmMyMod.lastEvent ?? null;
  return V._cmMyMod;
};

Add Function [setup.cm_mymod_level]:
setup.cm_mymod_level = function() {
  return setup.cm_mymod_init().level;
};

Add Function [setup.cm_mymod_level_add]:
setup.cm_mymod_level_add = function(n) {
  var state = setup.cm_mymod_init();
  state.level = Math.min(100, state.level + n);
};
```

StoryInit via Add Variable

Use Add Variable for simple scalar StoryInit declarations. For complex object initialization, put it in an Add Javascript: block with a guard, and call it from a StoryInit Append To Passage hook.

```
// Simple: Add Variable
Add Variable [myModEnabled]: true

// Complex: Add Javascript: + Append To Passage
Add Javascript:
setup.cm_mymod_on_story_init = function() {
  V._cmMyMod = V._cmMyMod ?? {};
  V._cmMyMod.level = V._cmMyMod.level ?? 0;
};

Append To Passage [StoryInit]:
With:
<<run setup.cm_mymod_on_story_init(>>
```

31. Event Authoring Best Practices

Event frequency and cooldowns

The chance field in Add Events: controls how often an event fires each evaluation cycle. A value of 0.05 means approximately 5% chance per cycle. But frequency alone is not enough -- any passage that modifies stats must also have a cooldown stored in state to prevent farming.

```
// Passage with stat change + cooldown guard:
EventMyModDorm

<<if (V._cmMyMod[V.eventnpc] ?? {}).lastEventDay === V.day>>
<<continuelink>><<stop>>
<</if>>
```

```
<<set _cm_state = V._cmMyMod[V.eventnpc] ?? {}>>
<<set _cm_state.lastEventDay = V.day>>
<<set V._cmMyMod[V.eventnpc] = _cm_state>>
...passage content with stat changes...
```

NPC guard is mandatory

Every event passage that accesses a named NPC must open with a null check on the NPC variable. Pattern:

```
<<if !$eventnpc>><<continuelink>><<stop>><</if>>
<<if typeof setup.people.get_attitude !== "function">><<continuelink>><<stop>><</if>>
... rest of event content ...
```

One-time passages must be idempotent

Passages that fire only once (intro passages, first-meeting passages, threshold-crossing passages) must open with a guard checking a stored flag and stop if already set. The flag must be set before any stat changes fire.

```
<<if V._cmMyMod.introFired>><<continuelink>><<stop>><</if>>
<<set V._cmMyMod.introFired = true>>
... intro content and stat changes below ...
```

Consequence branching

Any event or passage that represents a meaningful story beat must have at least two distinct outcome paths. A uniformly positive outcome from a repeatable event is bad design.

```
<<link "Accept warmly">>
<<goto "EventCMMymodWarm">>
<</link>>
<<link "Keep distance">>
<<goto "EventCMMymodCool">>
<</link>>
```

Stat change bounds

Single-event stat gains must be bounded. Recommended maximums per event: friendship +30, lust +20, romance +25, anger +25. Large one-time gains (first kiss, major story beat) may exceed these but must be flagged with a one-time-only guard.

32. Mod File Layout Conventions

Recommended directive ordering

Within a .mod file, directives should appear in this order for readability and to match the patcher's Phase 0-7 execution order:

- # 1. Header (all # comments -- description, identity, Provides:, changelog)
- # 2. Add Function blocks (all setup.cm_* functions)
 - Add Function [setup.cm_mymod_init]: ...
 - Add Function [setup.cm_mymod_check]: ...
- # 3. Add Javascript blocks (Macro.add, config objects, init hooks)

```

Add Javascript:
...

# 4. Add CSS blocks
Add CSS:
...

# 5. Add Passage blocks (all new passages)
Add Passage: EventCMMymodDorm
...

# 6. Add Events blocks
Add Events:
...

# 7. Append To Passage [Cheats]: (cheat menu hooks)
Append To Passage [Cheats]:
...

# 8. Insert After / Insert Before blocks (passage hooks)
Insert After [someAnchor]:
...

# 9. Replace In Passage blocks
# 10. Hook blocks
# 11. Replace: blocks targeting script sections LAST

```

Header order

The mod file header must always put the description/identity block first and the changelog below. Required order:

```

# Brief description of what this mod does
# By: CoderMaverick
# Filename: CM_Type.v1
# Target: COT v0.7.8c
# Requires: nothing (standalone)
#
# Provides:
#   Key passages / functions / systems this mod adds
#
# CM_ModName vX.Y.Z
# vX.Y.Z: What changed in this version.
# vX.Y.(Z-1): What changed in the previous version.

```

NOTE: Never start a header with the version number or changelog. The description and identity lines must come first. Submod authors open a mod file to understand what it provides -- they should find that immediately.

34. Advanced Patterns

Dispatch passages

When a mod needs to branch on sport, NPC archetype, relationship type, or any other enum, use a dispatch passage rather than inline branching in the event.

```
Add Passage: EventCMMymodDispatch
tags: nobr noevents
<<if _sport is "football"><<goto "EventCMMymodFootball"><</if>>
<<if _sport is "soccer"><<goto "EventCMMymodSoccer"><</if>>
<<if _sport is "basketball"><<goto "EventCMMymodBasketball"><</if>>
<<continuelink>>
```

Dispatch passages have no prose requirement -- they are routing logic only. They are exempt from the minimum viable event standard.

Phone icon injection

When adding a phone app icon, always use `_icons.push()` via Insert After on the PhoneMain passage. Never use Append To Passage [PhoneApps]: -- that passage does not exist. The passage is PhoneMain, not PhoneApps.

```
Insert After [PhoneMain]:
With:
<<if typeof setup.cm_mymod_init === "function">>
<<run _icons.push({
  name: "MyMod",
  img: "mods/mod-img/icon_mymod",
  fallbackImg: "ico_phone_reminders",
  dialog: "CMMymodPhonePanel",
  wide: true
})>>
<</if>>
```

Cooldown management pattern

For events that must not fire more than once per day per NPC:

```
Add Function [setup.cm_mymod_event_ready]:
setup.cm_mymod_event_ready = function(npc) {
  var state = (V._cmMyMod ?? {})[npc] ?? {};
  return state.lastEventDay !== V.day;
};

Add Function [setup.cm_mymod_event_mark]:
setup.cm_mymod_event_mark = function(npc) {
  V._cmMyMod = V._cmMyMod ?? {};
  V._cmMyMod[npc] = V._cmMyMod[npc] ?? {};
  V._cmMyMod[npc].lastEventDay = V.day;
};

# Event passage uses the guard at the top:
<<if !setup.cm_mymod_event_ready($eventnpc)>><<continuelink>><<stop>><</if>>
<<run setup.cm_mymod_event_mark($eventnpc)>>
... event content ...
```

Troubleshooting Guide for Mod Authors

"No match found" in the log

The most common failure. The Replace: anchor text does not exist in the game HTML exactly as written. Most frequent causes:

Encoding mismatch: the anchor is in a passage body (HTML-encoded) but you wrote raw Twine. Copy the anchor directly from the game HTML file opened in a text editor -- never reconstruct it from Twine source.

Script-block target: the anchor is in the JavaScript block and you HTML-encoded it (writing `<<` instead of `<<`). Script blocks store raw characters. v0.7.4 adds a two-pass search that fixes most of these automatically, but prefer Insert Into Object for JS object entries.

Whitespace difference: the game HTML may have different indentation than expected. Copy paste the exact lines including leading spaces.

Game version mismatch: the game was updated and the code the anchor targets no longer exists at that location. Run KittyValidate to identify which anchors fail before submitting a report.

"Ambiguous" in KittyValidate

The anchor was found more than once in the HTML. The directive will match the FIRST occurrence, which may be the wrong one. Fix options:

- Extend the anchor to include more surrounding context (1-2 extra lines unique to the target location).

- Switch from Insert After to Insert Into Function -- scope the match to the specific function body.

- Switch from Replace: to Replace In Passage -- scope the match to the specific passage.

The directive applied but the game behavior is wrong

KittyPatcher injects exactly what you write -- if the injected JS has a bug, the patcher injects the bug. Check the browser console (F12) for JavaScript errors after patching. Common issues:

- Undefined function: a typeof guard is missing. The function you call may not exist when your code runs (wrong load order, optional dep not loaded).

- Variable collision: your mod writes to a state variable that another mod also writes to. Check for namespace conflicts -- all CM mods use `V._cmModName` prefixed state.

- Double-execution: Add Javascript: setup code ran twice because it lacks a guard (if `!setup.myConfig`) `{...}`).

Hook body not firing

Hook: wraps assignment-style functions only -- `name = function(...) {}`. It skips named function declarations (`function name() {}`). Verify the target function uses assignment style. Also verify the function name exactly matches including the object path (`setup.people.generate` is different from `setup_people_generate`).

NOTE: v0.7.4 fixes cross-mod Hook conflicts. If two mods hook the same function, both bodies now fire. If only one fires, check that the other hook does not have a failing guard condition.

Passage shows but content is wrong or missing

If a passage was added via Add Passage: but shows empty or wrong content, check: (1) the NPC guard at the top of the passage -- if `$eventnpc` is null the passage stops immediately; (2) whether a state init guard cleared the passage early because a flag was already set; (3) whether another mod is also appending to this passage and overwriting expected state.

Conflict Resolution Strategies

When two mods conflict on the same target, the last mod in load order wins the Replace: battle. But there is almost always a way to avoid the conflict entirely through directive choice.

Replace Function conflicts

Both mods replace the same function body. Neither can win without losing the other's changes. The fix: both mods should use Hook instead. Each hooks the function independently. Both fire in load order. Neither replaces the other.

```
// CONFLICT: both mods replace the same function
// ModA:
Replace Function [setup.encounter_check]:
return modA_result;

// ModB:
Replace Function [setup.encounter_check]:
return modB_result; // wins, ModA lost

// FIX: both mods use Hook
// ModA:
Add Function [setup.modA_check]: ...
Hook [setup.encounter_check]:
after:
if (setup.modA_check()) _result = modA_result;

// ModB:
Add Function [setup.modB_check]: ...
Hook [setup.encounter_check]:
after:
if (setup.modB_check()) _result = modB_result;
```

Append To Passage conflicts

Two mods appending to the same passage is NOT a conflict -- both With: bodies apply in load order. The CONFLICT log entry is informational only. No fix needed unless you need to control which content appears first (use Priority: for that).

Replace: conflicts in passage content

Two mods replace the same line in the same passage. Fix: both mods should use Replace In Passage with different anchors targeting different parts of the passage, or one mod should use Append To Passage and post-process the output rather than replacing inline.

Add Events: duplicate registrations

Two mods register the same event passage in Add Events:. The event fires twice as often. Fix: one mod should remove its Add Events: entry and instead use a cross-mod compatibility submod that registers the event once with combined checkvar logic.

Extending Vanilla Game Systems

Specific patterns for safely extending the most commonly targeted vanilla systems without collision.

Extending setup.Events.db (event registration)

Always use Add Events: rather than Replace: or Insert After on the db array. Add Events: is Tier S and immune to surrounding code changes.

```
Add Events:
{
  passage: "EventMyCoolEvent",
  tags: ["dorm", "evening"],
  chance: 0.04,
  conditions: ["State.variables.myModEnabled"],
},
```

The conditions array takes JavaScript expression strings evaluated at event selection time. Keep them short and side-effect free.

Extending setup.inclinations or setup.clothes (data registries)

These are named JS objects in the script block. Use Insert Into Object rather than Replace: -- it anchors on the object name, not content lines.

```
Insert Into Object [setup.inclinations]:
"My New Inclination": {
  name: "My New Inclination",
  type: "sexual",
  description: "Describes the inclination.",
},
```

Extending the Person class (NPC properties)

To add properties to all NPCs, use Append Variable to Class and Append Function to Class. These are Tier S and detect ES6 vs prototype class syntax automatically.

```
Append Variable to Class [Person]:
this.myModProp = false;
```

```
Append Function to Class [Person]:
getMyModState() {
  return this.myModProp;
}
```

Hooking the day advance / WakeUp cycle

To run code at the start of each game day, hook setup.Time.do_new_day or Append To Passage [WakeUp]. Use Append To Passage for Twine passage code, Hook for JavaScript functions.

```
# Twine passage hook (runs as passage code):
Append To Passage [WakeUp]:
With:
<<run setup.mymod_daily_tick()>>

# JS function hook:
Hook [setup.Time.do_new_day]:
after:
if (typeof setup.mymod_daily_tick === "function") setup.mymod_daily_tick();
```

Adding phone app icons

Phone icons must be injected via Insert After on the PhoneMain passage using the `_icons.push()` pattern. The passage is PhoneMain, not PhoneApps -- that passage does not exist.

```
Insert After [PhoneMain]:
With:
<<if typeof setup.mymod_init === "function">>
<<run _icons.push({
  name: "My Mod",
  img: "mods/mod-img/icon_mymod",
  fallbackImg: "ico_phone_reminders",
  dialog: "CMMymodPhonePanel",
  wide: false
})>>
<</if>>
```

Dependency Graph Deep Dive

`cm_dependencies.json` controls mod load order. Understanding it lets you author mods that always load in the right sequence.

The dependency graph format

The file is a JSON object where each key is a mod filename and its value is an array of mods it depends on:

```
{
  "CM_FM.v1":      [],
  "CM_R0.v1":      [],
  "CM_FS.v1":      ["CM_FM.v1"],
  "CM_CE.v1":      ["CM_FM.v1"],
  "CM_R0Rivals.v1": ["CM_R0.v1", "CM_FM.v1"],
  "CM_CE_FS.v1":   ["CM_CE.v1", "CM_FS.v1"],
  "CM_SERivals.v2": ["CM_R0Rivals.v1"]
}
```

The patcher performs a topological sort: mods with no dependencies load first, then mods whose dependencies have all loaded, and so on. A mod is guaranteed to load after all its listed dependencies.

The Priority: field

When two mods are at the same dependency level (neither depends on the other), `Priority:` controls which loads first. Lower number = earlier.

```
# In mod file header:
# Priority: 500    <- loads earlier than default (1000)
# Priority: 1500   <- loads later than default
```

`Priority:` is only a tiebreaker within a dependency level. It cannot override actual dependency ordering -- a child mod always loads after its parents regardless of `Priority:`.

Cycles in the dependency graph

If mod A depends on mod B and mod B depends on mod A, the patcher detects the cycle, logs an error to FailsPatchLog.txt, and falls back to Priority:/filename order for the mods involved. The patch run does not abort. Cycles are always a mod authoring error -- restructure the dependency to eliminate them.

Mods not in cm_dependencies.json

A mod does not need to be in cm_dependencies.json to load. Mods absent from the file load in filename order after all dependency-graph mods have loaded. This makes cm_dependencies.json forward-compatible -- new mods can be added to the mods/ folder without updating the file.

NOTE: The Requires: field in the mod file header emits a warning if a named mod is absent, but does not enforce load order. Only cm_dependencies.json enforces ordering.

Writing Version-Resilient Mods

The goal is to write a mod that keeps working across game updates with zero or minimal porting work. This section explains how.

The golden rule: anchor on names, not content

Every Tier S and Tier 1 directive anchors on a name (passage name, function name, object name). These survive game updates as long as the name is not changed. Tier 4-5 directives anchor on specific lines of content -- any line change in a game update breaks them.

The single most impactful choice in mod authoring: put all your logic in Add Function blocks (Tier S), then call them from Hook (Tier 2). The Add Function never needs porting. The Hook anchor is a function name -- it only breaks if the game renames the function. Content line changes in the function body are invisible to Hook.

Minimizing Replace: surface area

When Replace: is unavoidable, keep the anchor as short as possible -- 2-3 lines maximum. A longer anchor has more lines that can drift. If you need to modify 10 lines of a passage, consider: can you add code that checks state after the vanilla code runs instead of replacing the vanilla code?

Using Insert Into Function over Insert After

Insert After searches the entire HTML for the first match. Insert Into Function scopes the search to one function body. If the same anchor string appears in two functions and the game adds a third in an update, Insert After might match the wrong one. Insert Into Function is immune to new occurrences of the same string elsewhere in the HTML.

Guard your cross-mod calls everywhere

typeof checks ensure your mod degrades gracefully when optional dependencies are absent. A mod that crashes when an optional parent is missing is harder to maintain than one that silently skips optional behavior.

```
// Always guard cross-mod calls:  
Add Function [setup.mymod_fame_add]:
```

```

setup.mymod_fame_add = function(cat, n) {
  if (typeof setup.cm_fame_add !== "function") return;
  setup.cm_fame_add(cat, n);
};

```

Document what your mod touches

The Provides: block in the mod header tells other mod authors what your mod exports. A submod author who needs to Append To Passage on one of your hook-surface passages needs to know its name. If it is not documented, they have to read your entire mod file.

```

# Provides:
#   setup.mymod_init()           -- initializes per-save state
#   setup.mymod_level(npc)      -- returns current level for npc
#   EventCMMMyModCleanup        -- hook surface: append post-encounter cleanup
#   EventCMMMyModDailyTick      -- hook surface: append daily state updates

```

27. Quick Reference

All directive syntax

```

// TIER S -- additive, zero porting cost
Add Passage: PassageName (or full <tw-passagedata> format)
Add Javascript: Add CSS: Add Events: Add Function [name]: Add Variable [name]: value
Rename Passage [OldName]: NewName Move Passage [OldName] To [NewName]:
Add Tag To Passage [Name]: tag Remove Tag From Passage [Name]: tag

// TIER 1 -- passage name anchor
Prepend To Passage [Name]: With: content
Append To Passage [Name]: With: content

// TIER 2 -- near-zero porting cost
Hook [setup.existingFunc]: before: | after: | around: (v0.7.0)
Insert Before [anchorString]: With: code Insert After [anchorString]: With: code

// TIER 2.5 -- scoped, moderate porting cost
Insert Into Function [setup.func]: anchor With: code after anchor
Replace In Function [setup.func]: anchor With: replacement
Delete In Passage [PassageName]: text to delete

// TIER 3 -- moderate porting cost
Replace Function [setup.existingFunc]: // complete body, no braces
Replace Function Signature [setup.func]: newParam1, newParam2

// TIER 4-5 -- higher porting cost
Replace In Passage [PassageName]: anchor With: replacement
Delete Span [start anchor] To [end anchor]:
ReplaceReg [(?s)prefix(CAPTURE)suffix]: With{1}: replacement for group 1
Replace: <exact HTML> With: [raw Twine -- auto-escaped in v0.7.0]

// STRUCTURAL -- anchors on name
Append Array [arrayName In Function [setup.init]]: "new item",
Append Variable to Class [ClassName]: this.newProp = 0;
Append Function to Class [ClassName]: newMethod() { return this.newProp; }
Insert Into Array [arrayName]: "item",

```

```

Insert Into Object [objectName]: key: value,
Delete Block [PassageName]: // no duplicate check

// GUARDS (v0.7.0: 12 total)
IfExists [s]: IfNotExists [s]: IfRegExists [p]: IfRegNotExists [p]:
IfModLoaded [f]: IfModNotLoaded [f]:
IfPassageExists [n]: IfPassageNotExists [n]:
IfFunctionExists [n]: IfFunctionNotExists [n]:
IfPassageContains [Name|text]: IfPassageHasTag [Name|tag]: <- v0.7.0

```

Porting cost at a glance

Tier	Directives	Fails when
S	Add Passage, Add JS/CSS/Events, Add Function, Add Variable, Rename, Move, Tag ops	Never
1	Prepend/Append To Passage	Passage renamed or removed
2	Hook, Insert Before, Insert After	Function/anchor renamed or removed
2.5	Insert Into Function, Replace In Function, Replace Func Sig, Delete In Passage	Name or anchor within changes
3	Replace Function	Function renamed; body drift silent
4	Replace In Passage, Delete Span	Passage renamed or anchor changes
5	ReplaceReg, Replace:	Any targeted line changes
S	Append Array/Var/Func, Insert Into Array/Object, Delete Block	Name renamed or removed

Log files at a glance

File	Contents
ModPatchLog.txt	Every attempt, success, skip, and failure with the search key
FailsPatchLog.txt	Only failures and conflict warnings
MainPatchLog.txt	Summary: counts, successful mods, failed mods, timing
CustomLog.txt	Custom mod messages
DiffAssist_<mod>.txt	Per-failure diff report (Modder Edition only)
PortDraft_<mod>.mod	Auto-updated mod file draft (Modder Edition only)

Authoring Session Workflow Reference

A complete authoring session for a new mod from start to release:

Step	Command	Notes
1. Scaffold	<code>python KittyScaffold.py --name CM_MyMod --type main --with-tests</code>	Generates starter .mod file for the new mod
2. Author	Text editor	Fill in Add Function, Add Passage, Hook blocks per resilience tier hierarchy. Disable cache: [cache] enabled = false.
3. Validate	<code>python KittyValidate.py game.html MyMod.mod</code>	Resolve all will-fail and ambiguous statuses before continuing
4. Patch	<code>python KittyPatcher.py</code>	Apply mod. Check ModPatchLog.txt and FailsPatchLog.txt.
5. Game test	Launch game	Test runtime behavior, check browser console for JS errors